In [4]:
```
import matplotlib
from matplotlib import pyplot as plt
%matplotlib inline

matplotlib.rc('figure', figsize=(12, 5))
font = {'family' : 'normal',
        'weight' : 'bold',
        'size'   : 16}

matplotlib.rc('font', **font)
```

In [5]:
```
# from http://zyxue.github.io/2017/09/21/python-timeit-decorator.html
import datetime
import time
from functools import wraps
from typing import Any, Callable


def timeit(func: Callable[..., Any]) -> Callable[..., Any]:
    """Times a function, usually used as decorator"""

    @wraps(func)
    def timed_func(*args: Any, **kwargs: Any) -> Any:
        """Returns the timed function"""
        start_time = time.time()
        result = func(*args, **kwargs)
        elapsed_time = datetime.timedelta(seconds=(time.time() - start_time))
        print("time spent on %s: %s", func.__name__, elapsed_time)
        return result

    return timed_func
```

In [6]:
```
import seaborn as sns; sns.set()

def visualize_solution(solution):
    """visualize a board of arbitrary size"""
    if isinstance(solution, dict):
        solution = list(solution.values())
    if isinstance(solution, list):
        solution = np.array(solution)

    n = int(np.sqrt(solution.size))
    plt.figure(figsize=(5, 3))
    sns.heatmap(
        solution.reshape((n, n)),
        cbar=False,
        cmap="YlGnBu",
        annot=True
    )
    plt.show()
```

In [7]:
```
"""
From the dd examples on github:
https://github.com/tulip-control/dd/blob/0f6d16483cc13078edebac9e89d1d4b99d22991e/examples/queens

N-Queens problem using one-hot encoding.
Reference
=========
Henrik R. Andersen
```

```python
    "An introduction to binary decision diagrams"
    Lecture notes for "Efficient Algorithms and Programs", 1999
    The IT University of Copenhagen
    Sec.6.1
"""
import pickle
import time
from dd import bdd as _bdd
from omega.logic.syntax import conj, disj


def solve_queens(n):
    """Return set of models for the `n`-queens problem.
    @rtype: `int`, `BDD`
    """
    vrs = [_var_str(i, j) for i in range(n) for j in range(n)]
    bdd = _bdd.BDD()
    bdd.declare(*vrs)
    s = queens_formula(n)
    u = bdd.add_expr(s)
    return u, bdd


def queens_formula(n):
    """Return a non-trivial propositional formula for the problem."""
    # i = row index
    # j = column index
    present = at_least_one_queen_per_row(n)
    rows = at_most_one_queen_per_line(True, n)
    cols = at_most_one_queen_per_line(False, n)
    slash = at_most_one_queen_per_diagonal(True, n)
    backslash = at_most_one_queen_per_diagonal(False, n)
    s = conj([present, rows, cols, slash, backslash])
    return s


def at_least_one_queen_per_row(n):
    """Return formula as `str`."""
    c = list()
    for i in range(n):
        xijs = [_var_str(i, j) for j in range(n)]
        s = disj(xijs)
        c.append(s)
    return conj(c)


def at_most_one_queen_per_line(row, n):
    """Return formula as `str`.
    @param row: if `True`, then constrain rows, else columns.
    """
    c = list()
    for i in range(n):
        if row:
            xijs = [_var_str(i, j) for j in range(n)]
        else:
            xijs = [_var_str(j, i) for j in range(n)]
        s = mutex(xijs)
        c.append(s)
    return conj(c)


def at_most_one_queen_per_diagonal(slash, n):
```

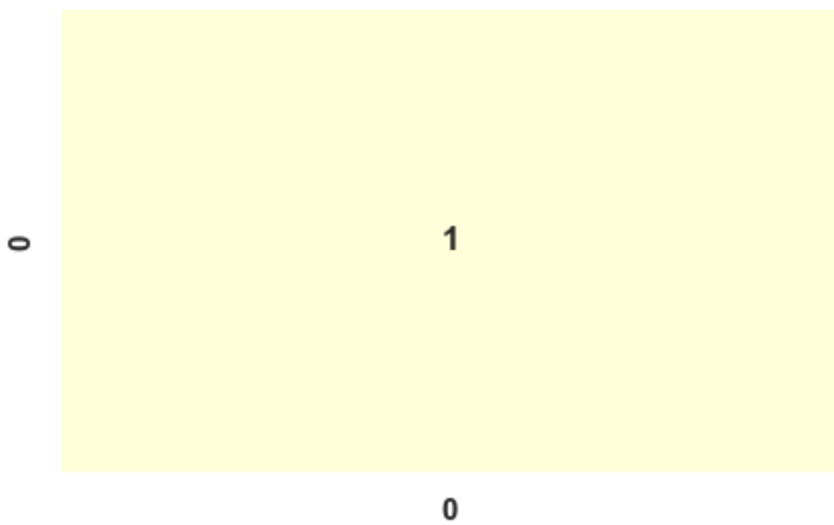
```python
    """Return formula as `str`.
    @param slash: if `True`, then constrain anti-diagonals,
        else diagonals.
    """
    c = list()
    if slash:
        a = -n
        b = n
    else:
        a = 0
        b = 2 * n
    for k in range(a, b):
        if slash:
            ij = [(i, i + k) for i in range(n)]
        else:
            ij = [(i, k - i) for i in range(n)]
        ijs = [(i, j) for i, j in ij if 0 <= i < n and 0 <= j < n]
        if not ij:
            continue
        xijs = [_var_str(i, j) for i, j in ijs]
        s = mutex(xijs)
        c.append(s)
    return conj(c)


def mutex(v):
    """Return formula for at most one variable `True`.
    @param v: iterable of variables as `str`
    """
    v = set(v)
    c = list()
    for x in v:
        rest = disj(y for y in v if y != x)
        s = '{x} -> !({rest})'.format(x=x, rest=rest)
        c.append(s)
    return conj(c)


def _var_str(i, j):
    """Return variable for occupancy of cell at {row: i, column: j}."""
    return 'x{i}{j}'.format(i=i, j=j)


def benchmark(n):
    """Run for `n` queens and print statistics."""
    t0 = time.time()
    u, bdd = solve_queens(n)
    t1 = time.time()
    dt = t1 - t0

    for i, d in enumerate(bdd.pick_iter(u)):
        if len(d) > 0:
            visualize_solution(d)
            break
    n_solutions = bdd.count(u)

    s = (
        '------\n'
        'queens: {n}\n'
        'time: {dt} (sec)\n'
        'node: {u}\n'
        'total nodes: {k}\n'
```

```
        'number solutions: {n_solutions}\n'
        '------\n'
    ).format(
        n=n, dt=dt, u=u, k=len(bdd),
        n_solutions=n_solutions,
    )
    print(s)
    return dt
```

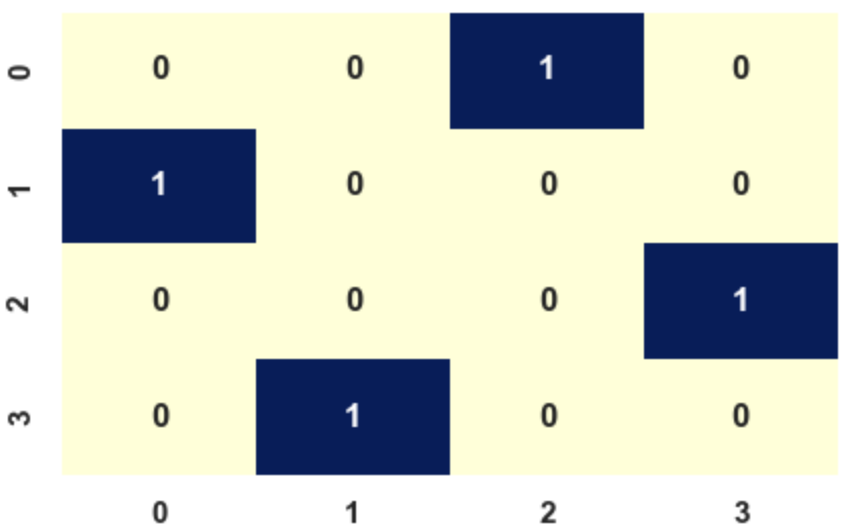
In [9]:

```
import numpy as np

n_max = 9
fname = 'dd_times.p'
times = dict()
for n in range(n_max + 1):
    t = benchmark(n)
    times[n] = t
f = open(fname, 'wb')
pickle.dump(times, f)
```

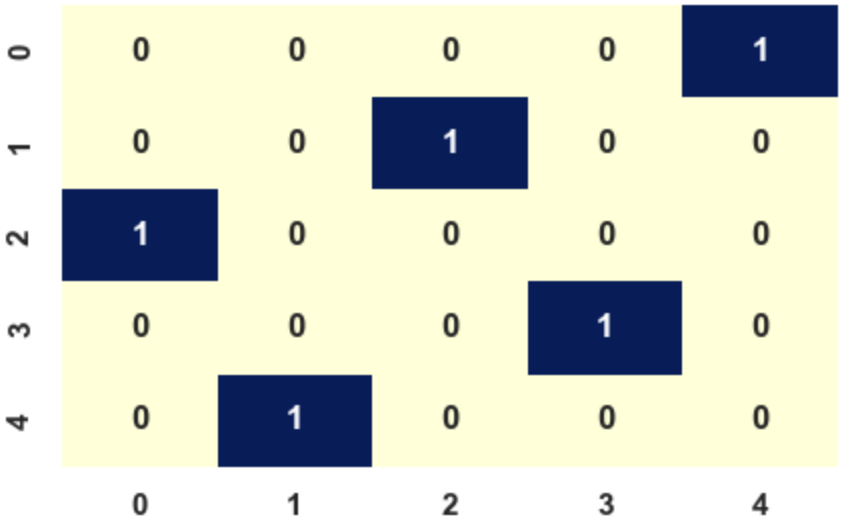
```
------
queens: 0
time: 0.00020003318786621094 (sec)
node: 1
total nodes: 1
number solutions: 1
------
```



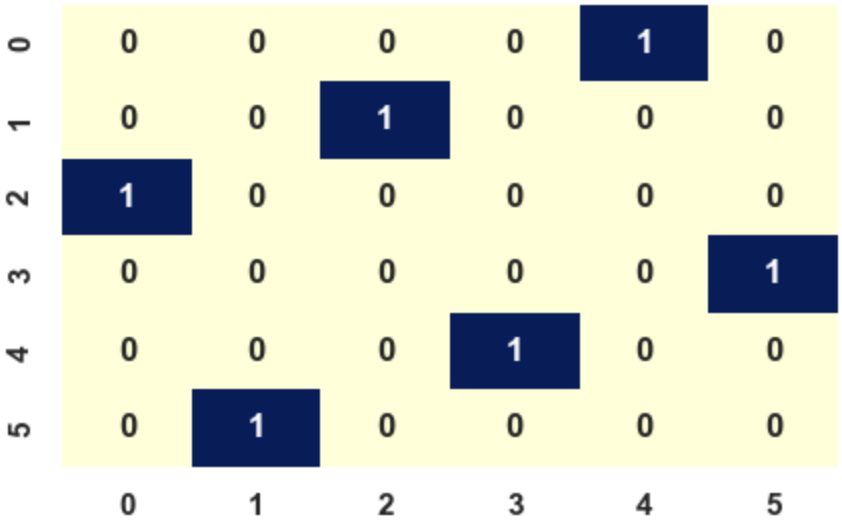
```
------
queens: 1
time: 0.0006594657897949219 (sec)
node: 2
total nodes: 2
number solutions: 1
------

------
queens: 2
time: 0.0025773048400878906 (sec)
node: -1
total nodes: 27
number solutions: 0
------

------
queens: 3
time: 0.010674715042114258 (sec)
node: -1
total nodes: 212
number solutions: 0
------
```



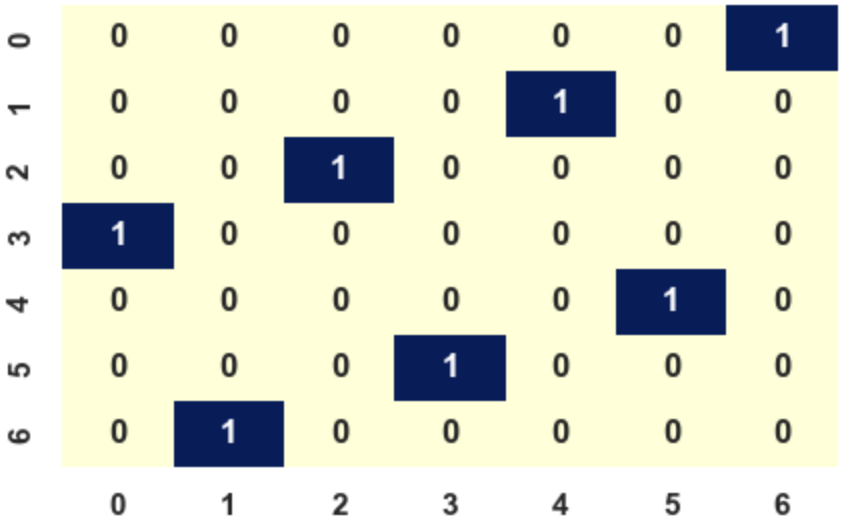

```
------
queens: 4
time: 0.029198884963989258 (sec)
node: -915
total nodes: 915
number solutions: 2
------
```



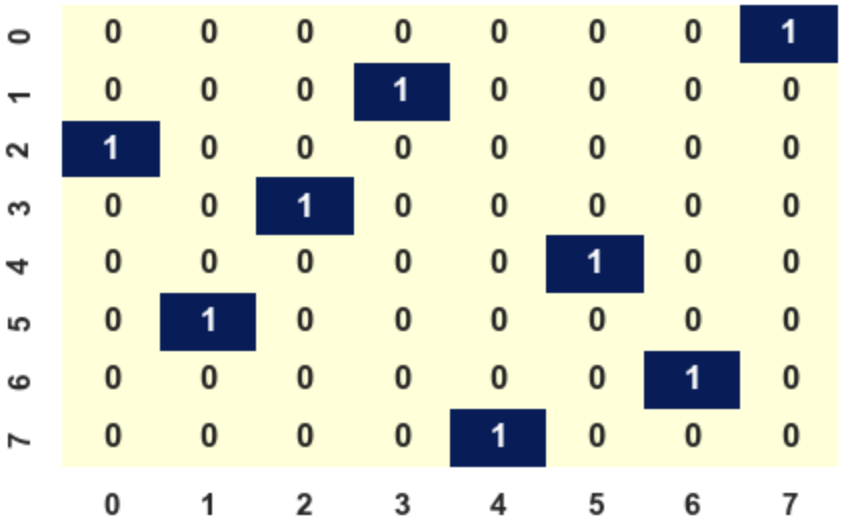

```
------
queens: 5
time: 0.07702255249023438 (sec)
node: -3636
total nodes: 3636
number solutions: 10
------
```



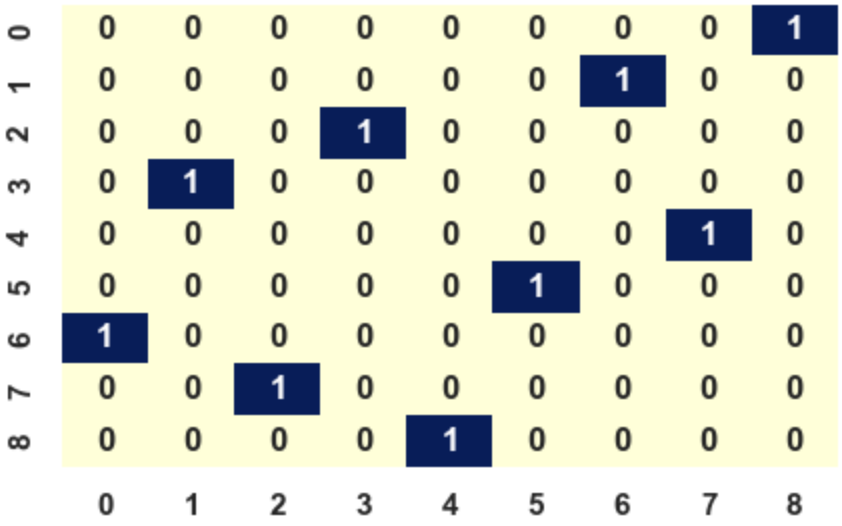

```
------
queens: 6
time: 0.4241962432861328 (sec)
node: -13460
total nodes: 13460
number solutions: 4
------
```




```
------
queens: 7
time: 0.8217620849609375 (sec)
node: -55840
total nodes: 55840
number solutions: 40
------
```




```
------
queens: 8
time: 5.615418434143066 (sec)
node: -250824
total nodes: 250824
number solutions: 92
------
```



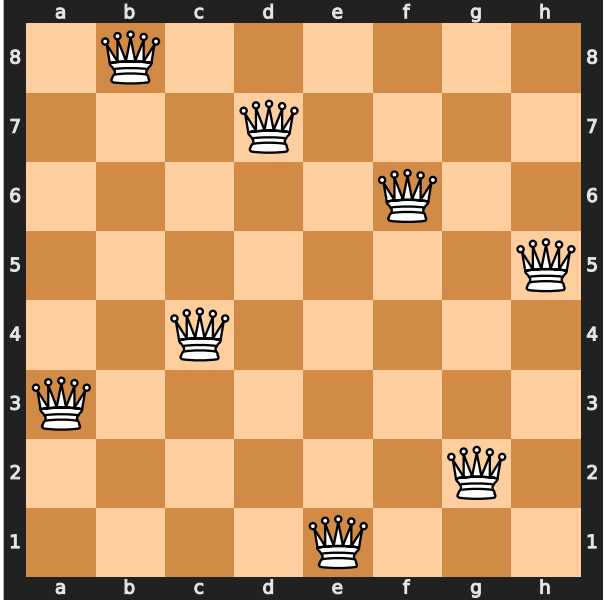

```
------
queens: 9
time: 25.763606071472168 (sec)
node: -1200102
total nodes: 1200102
number solutions: 352
------
```

In [47]:

```python
def plot_scores(scores):
    plt.plot(
        list(range(len(scores))),
        scores,
        '--o',
    )
    plt.ylabel('score')
    plt.xlabel('iterations')
    plt.grid(True)
    plt.show()   # ← REQUIRED
```

In [11]:

```python
import chess
import chess.svg
from IPython.display import display

def queen_to_str(q, N=8):
    prev = str(q) if q > 0 else ''
    post = str((N-1)-q) if q < (N-1) else ''
    return prev + 'Q' + post

assert queen_to_str(1) == '1Q6'
assert queen_to_str(0) == 'Q7'
assert queen_to_str(7) == '7Q'

def show_board(queens):
    """This only works for 8 queens..."""
    fen = '/'.join([queen_to_str(q) for q in queens])
    display(chess.svg.board(board=chess.Board(fen), size=300))

solution = np.array([2, 4, 6, 8, 3, 1, 7, 5]) - 1
show_board(solution)
```




In [45]:
```
import numpy as np

def cost_function(props):
    """Given a props vector from a particle,
    a vector of queens' positions, how right are we?
    """
    res = 0
    for i1, q1 in enumerate(props[:-1]):
        for i2, q2 in enumerate(props[i1+1:], i1+1):
            if (q1 != q2) and (abs(i1 - i2) != abs(q1 - q2)):
                res += 1
    return res
```

In [13]:
```
import random
from typing import Optional, List, Tuple
import numpy as np

class Chromosome:
    def __init__(self, configuration: Optional[List]=None, nq: Optional[int]=None):
        """make sure either nq or configuration is given!
        """
        assert not (nq is None and configuration is None)
        if configuration is None:
            self.nq = nq
            self.max_fitness = np.sum(np.arange(nq))
            self.configuration = [
                random.randint(1, nq) for _ in range(nq)
            ]
        else:
            self.configuration = configuration
            self.nq = len(configuration)
            self.max_fitness = np.sum(np.arange(self.nq))

    def fitness(self):
        return cost_function(self.configuration) / self.max_fitness

    def mutate(self):
        """A mutation: randomly changing a value
        inside a chromosome"""
        ind = random.randint(0, self.nq-1)
        val = random.randint(1, self.nq)
        self.configuration[ind] = val
```

```python
class GeneticQueen:
    def __init__(self, nq, population_size=20, mutation_prob=0.5):
        self.nq = nq
        self.population_size = population_size
        self.mutation_prob = mutation_prob
        self.population = [Chromosome(nq=nq) for _ in range(population_size)]
        self.solution = None
        self.best_fitness = None

    def iterate(self):
        """create a new generation
        """
        new_population = []
        best_fitness = -1
        for i in range(len(self.population)):
            p1, p2 = self.get_parents()
            child = Chromosome(self.cross_over(p1, p2))

            if random.random() < self.mutation_prob:
                child.mutate()

            new_population.append(child)
            fit = child.fitness()
            if fit > best_fitness:
                best_fitness = fit
            if fit == 1:
                self.solution = child
                break

        self.best_fitness = best_fitness
        self.population = new_population

    def cross_over(self, p1, p2):
        """cross_over between two chromosomes
        """
        return [
            yi
            if random.random() > 0
            else xi
            for xi, yi in zip(
                p1.configuration,
                p2.configuration
            )
        ]

    def get_parents(self) -> Tuple[Chromosome, Chromosome]:
        """choose parents to reproduce;
        fittest more likely to be chosen
        """
        weights = [chrom.fitness() for chrom in self.population]
        return tuple(
            random.choices(
                self.population,
                weights=weights,
                k=2
            )
        )

    def visualize_solution(self):
        """visualize a board of arbitrary size"""
        if self.nq == 8:
```

```
            show_board(np.array(self.solution.configuration)-1)
        else:
            board = np.zeros(shape=(self.nq, self.nq))
            for row, col in enumerate(self.solution.configuration):
                board[row, col-1] = 1
            plt.figure(figsize=(5, 3))
            sns.heatmap(
                board,
                cbar=False,
                cmap="YlGnBu",
                annot=True
            )
            plt.show()

@timeit
def ga_solver(nq):
    fitness_trace = []
    gq = GeneticQueen(nq=nq)
    generation = 0
    while not gq.solution:
        gq.iterate()
        if (generation % 100) == 0:
            print('Generation {}'.format(generation))
            print('Maximum Fitness = {:.3f}'.format(gq.best_fitness))
        fitness_trace.append(gq.best_fitness)
        generation += 1

    gq.visualize_solution()
    return fitness_trace
```

In [14]:

```
fitness_trace = ga_solver(8)
```

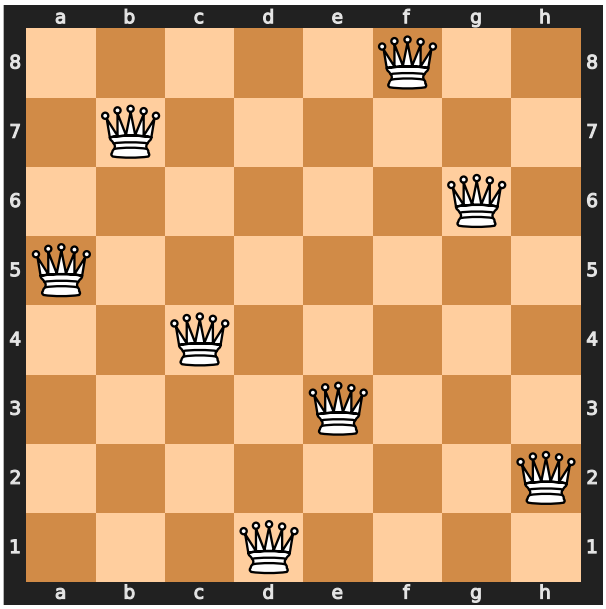
```
Generation 0
Maximum Fitness = 0.893
Generation 100
Maximum Fitness = 0.821
Generation 200
Maximum Fitness = 0.857
Generation 300
Maximum Fitness = 0.857
Generation 400
Maximum Fitness = 0.964
Generation 500
Maximum Fitness = 0.821
Generation 600
Maximum Fitness = 0.893
Generation 700
Maximum Fitness = 0.821
Generation 800
Maximum Fitness = 0.893
Generation 900
Maximum Fitness = 0.893
Generation 1000
Maximum Fitness = 0.821
Generation 1100
Maximum Fitness = 0.929
Generation 1200
Maximum Fitness = 0.893
Generation 1300
Maximum Fitness = 0.857
Generation 1400
Maximum Fitness = 0.821
Generation 1500
Maximum Fitness = 0.857
Generation 1600
Maximum Fitness = 0.893
Generation 1700
Maximum Fitness = 0.893
Generation 1800
Maximum Fitness = 0.893
Generation 1900
Maximum Fitness = 0.857
Generation 2000
Maximum Fitness = 0.857
Generation 2100
Maximum Fitness = 0.893
Generation 2200
Maximum Fitness = 0.821
Generation 2300
Maximum Fitness = 0.857
Generation 2400
Maximum Fitness = 0.821
Generation 2500
Maximum Fitness = 0.786
Generation 2600
Maximum Fitness = 0.929
Generation 2700
Maximum Fitness = 0.857
Generation 2800
Maximum Fitness = 0.893
Generation 2900
Maximum Fitness = 0.821
Generation 3000
Maximum Fitness = 0.893
```

```
Generation 3100
Maximum Fitness = 0.893
Generation 3200
Maximum Fitness = 0.857
Generation 3300
Maximum Fitness = 0.821
Generation 3400
Maximum Fitness = 0.857
Generation 3500
Maximum Fitness = 0.929
Generation 3600
Maximum Fitness = 0.857
Generation 3700
Maximum Fitness = 0.821
Generation 3800
Maximum Fitness = 0.929
Generation 3900
Maximum Fitness = 0.857
Generation 4000
Maximum Fitness = 0.821
Generation 4100
Maximum Fitness = 0.821
Generation 4200
Maximum Fitness = 0.893
Generation 4300
Maximum Fitness = 0.893
Generation 4400
Maximum Fitness = 0.929
Generation 4500
Maximum Fitness = 0.893
Generation 4600
Maximum Fitness = 0.893
Generation 4700
Maximum Fitness = 0.893
Generation 4800
Maximum Fitness = 0.857
Generation 4900
Maximum Fitness = 0.857
Generation 5000
Maximum Fitness = 0.857
Generation 5100
Maximum Fitness = 0.857
Generation 5200
Maximum Fitness = 0.893
Generation 5300
Maximum Fitness = 0.893
Generation 5400
Maximum Fitness = 0.857
Generation 5500
Maximum Fitness = 0.857
Generation 5600
Maximum Fitness = 0.821
Generation 5700
Maximum Fitness = 0.857
Generation 5800
Maximum Fitness = 0.893
Generation 5900
Maximum Fitness = 0.929
Generation 6000
Maximum Fitness = 0.786
Generation 6100
Maximum Fitness = 0.893
```

```
Generation 6200
Maximum Fitness = 0.893
Generation 6300
Maximum Fitness = 0.857
Generation 6400
Maximum Fitness = 0.929
Generation 6500
Maximum Fitness = 0.857
Generation 6600
Maximum Fitness = 0.857
Generation 6700
Maximum Fitness = 0.821
Generation 6800
Maximum Fitness = 0.857
Generation 6900
Maximum Fitness = 0.893
Generation 7000
Maximum Fitness = 0.857
Generation 7100
Maximum Fitness = 0.821
Generation 7200
Maximum Fitness = 0.893
Generation 7300
Maximum Fitness = 0.857
Generation 7400
Maximum Fitness = 0.893
Generation 7500
Maximum Fitness = 0.857
Generation 7600
Maximum Fitness = 0.857
Generation 7700
Maximum Fitness = 0.893
Generation 7800
Maximum Fitness = 0.893
Generation 7900
Maximum Fitness = 0.893
Generation 8000
Maximum Fitness = 0.893
Generation 8100
Maximum Fitness = 0.893
Generation 8200
Maximum Fitness = 0.857
Generation 8300
Maximum Fitness = 0.857
Generation 8400
Maximum Fitness = 0.821
Generation 8500
Maximum Fitness = 0.786
Generation 8600
Maximum Fitness = 0.821
Generation 8700
Maximum Fitness = 0.929
Generation 8800
Maximum Fitness = 0.893
Generation 8900
Maximum Fitness = 0.893
Generation 9000
Maximum Fitness = 0.893
Generation 9100
Maximum Fitness = 0.893
Generation 9200
Maximum Fitness = 0.893
```

```
Generation 9300
Maximum Fitness = 0.857
Generation 9400
Maximum Fitness = 0.857
Generation 9500
Maximum Fitness = 0.857
Generation 9600
Maximum Fitness = 0.857
Generation 9700
Maximum Fitness = 0.857
Generation 9800
Maximum Fitness = 0.929
Generation 9900
Maximum Fitness = 0.893
Generation 10000
Maximum Fitness = 0.857
Generation 10100
Maximum Fitness = 0.857
Generation 10200
Maximum Fitness = 0.857
Generation 10300
Maximum Fitness = 0.857
Generation 10400
Maximum Fitness = 0.821
Generation 10500
Maximum Fitness = 0.893
Generation 10600
Maximum Fitness = 0.893
Generation 10700
Maximum Fitness = 0.857
Generation 10800
Maximum Fitness = 0.893
Generation 10900
Maximum Fitness = 0.893
Generation 11000
Maximum Fitness = 0.893
Generation 11100
Maximum Fitness = 0.929
Generation 11200
Maximum Fitness = 0.893
Generation 11300
Maximum Fitness = 0.821
Generation 11400
Maximum Fitness = 0.857
Generation 11500
Maximum Fitness = 0.893
Generation 11600
Maximum Fitness = 0.821
Generation 11700
Maximum Fitness = 0.857
Generation 11800
Maximum Fitness = 0.857
Generation 11900
Maximum Fitness = 0.857
Generation 12000
Maximum Fitness = 0.857
Generation 12100
Maximum Fitness = 0.857
Generation 12200
Maximum Fitness = 0.857
Generation 12300
Maximum Fitness = 0.857
```

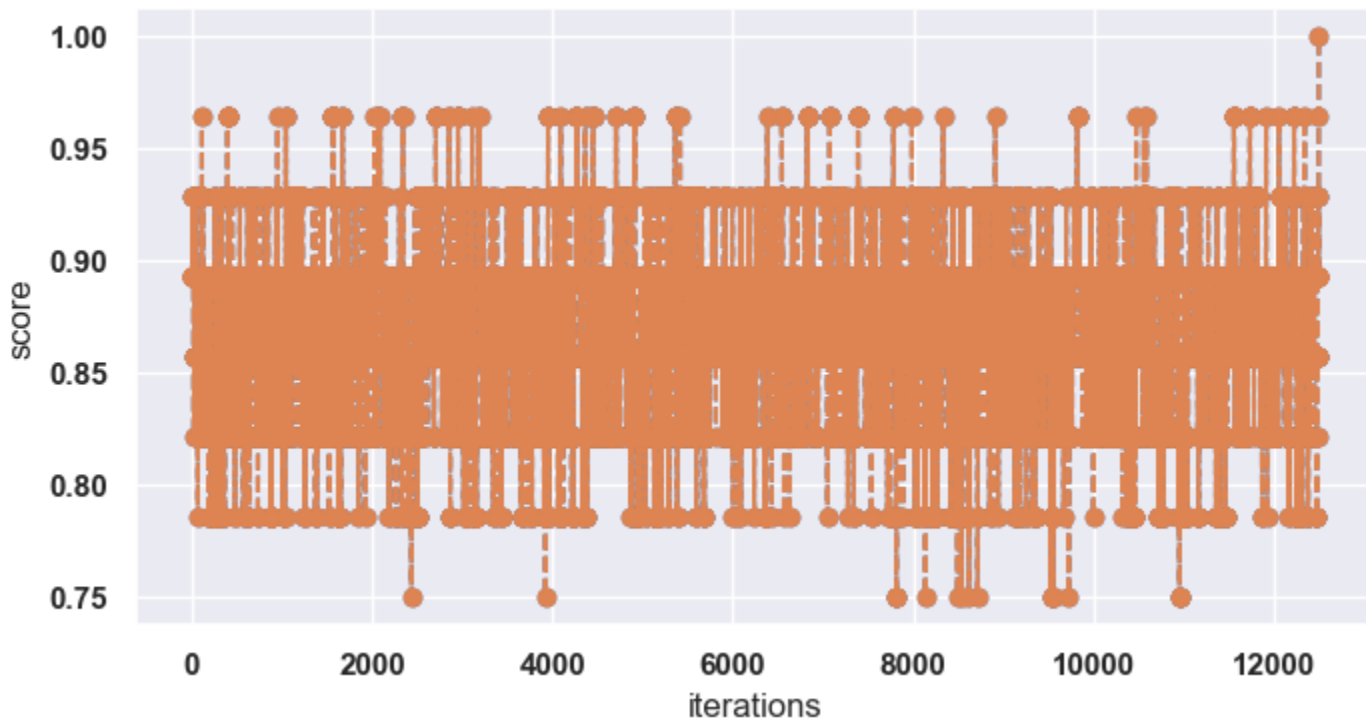
```
Generation 12400
Maximum Fitness = 0.893
```




```
time spent on %s: %s ga_solver 0:00:40.824682
```

In [48]:
```
plot_scores(fitness_trace)
```

```
<Figure size 800x400 with 0 Axes>
<Figure size 800x400 with 0 Axes>
<Figure size 800x400 with 0 Axes>
```




In [17]:
```
NQUEENS = 8
SOLVED_COST = np.sum(np.arange(NQUEENS))
assert SOLVED_COST == 28
```

In [18]:
```
class Particle:
    best_fitness: int = 0

    def __init__(
        self, N=None, props=None,
        velocities=None
    ):
```

```
        """Either specify N for randomization,
        or props and velocities
        """
        if props is None:
            # random initialization
            self.current_particle = np.random.randint(0, N-1, N)
            self.best_state = np.random.randint(0, N-1, N)
            self.velocities = np.random.uniform(-(N-1), N-1, N)
        else:
            self.current_particle = props
            self.best_state = props
            self.velocities = velocities

        self.best_fitness = cost_function(self.best_state)

    def set_new_best(self, props: List[int], new_fitness: int):
        self.best_state = props
        self.best_fitness = new_fitness

    def __repr__(self):
        return f'{self.__class__.__name__}(\n' +\
            f'\tcurrent_particle={self.current_particle}\n'  +\
            f'\best_state={self.best_state}\n'  +\
            f'\tvelocities={self.velocities}\n'  +\
            f'\best_fitness={self.best_fitness}\n'  +\
            ')'
```

In [19]:
```
def init_particles(N: int, n_particles: int):
    return [Particle(N=N) for i in range(n_particles)]

particles = init_particles(8, 20)
```

In [20]:
```
particles[0]
```

Out[20]:
```
Particle(
        current_particle=[2 0 2 5 0 0 4 4]
est_state=[5 1 6 4 2 4 5 3]
        velocities=[ 1.90116855  4.91670518 -6.47797287 -0.27444911 -6.78474275 -2.76272234
  2.96179966  0.99969917]
est_fitness=25
)
```

In [21]:
```
particles[0].current_particle
```

Out[21]:
```
array([2, 0, 2, 5, 0, 0, 4, 4], dtype=int32)
```

In [22]:
```
particles[0].current_particle
```

Out[22]:
```
array([2, 0, 2, 5, 0, 0, 4, 4], dtype=int32)
```

In [23]:
```
cost_function(np.array([7, 9, 3, 8, 2, 4, 6, 1]))
```

Out[23]:
```
28
```

In [24]:
```
particles[0]
```

Out[24]:
```
Particle(
        current_particle=[2 0 2 5 0 0 4 4]
est_state=[5 1 6 4 2 4 5 3]
        velocities=[ 1.90116855  4.91670518 -6.47797287 -0.27444911 -6.78474275 -2.76272234
  2.96179966  0.99969917]
est_fitness=25
)
```

In [25]:
```
# cdef int[:] get_best_particle(list particles):
# def get_best_particle(particles: List[Particles]) -> Particle:

def get_best_particle(particles):
    best_particle = 0  # int
    best_score = -1  # int
    score = -1  # int
    for i, particle in enumerate(particles):
        score = cost_function(particle.current_particle)
        if score > best_score:
            best_score = score
            best_ind = i
    return particles[best_ind].current_particle, best_score
```

In [26]:
```
particles = particles[:-1]
```

In [27]:
```
particles.append(Particle(props=solution))
```

In [28]:
```
get_best_particle(particles)  #  == (solution, 28)
```

Out[28]:
```
(array([1, 3, 5, 7, 2, 0, 6, 4]), 28)
```

In [29]:
```
particles[0]
```

Out[29]:
```
Particle(
        current_particle=[2 0 2 5 0 0 4 4]
est_state=[5 1 6 4 2 4 5 3]
        velocities=[ 1.90116855  4.91670518 -6.47797287 -0.27444911 -6.78474275 -2.76272234
  2.96179966  0.99969917]
est_fitness=25
)
```

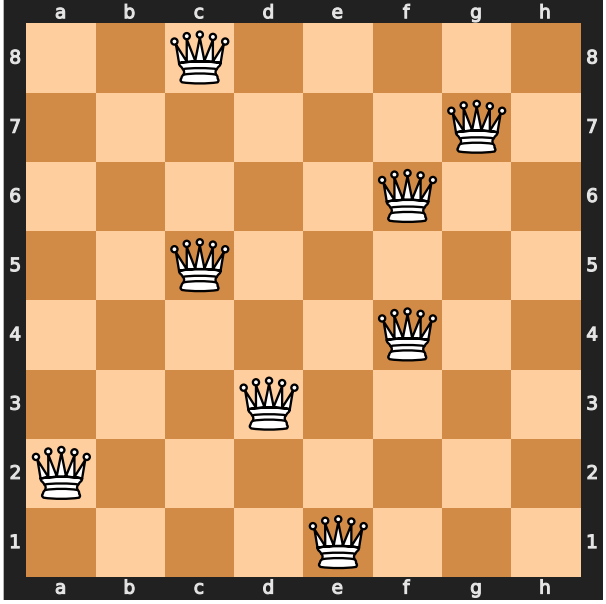
In [30]:
```
# good explanation and pseudocode https://en.wikipedia.org/wiki/Particle_swarm_optimization
```

In [31]:
```
def particle_swarm_optimization(
    N: int, omega: float, phip: float, phig: float,
    n_particles: int, visualize=False, max_iteration=999999
) -> List[int]:
    """Particle Swarm Optimization

    Parameters:
    -----------
    N - number of queens
    omega - a decay parameter
    phip - controls the contribution of the local search
    phig - controls the contribution of the global search
    n_particles - the number of particles
    visualize - if the solutions should be visualized by images
    max_iteration - for early stopping

    Returns:
    --------
```

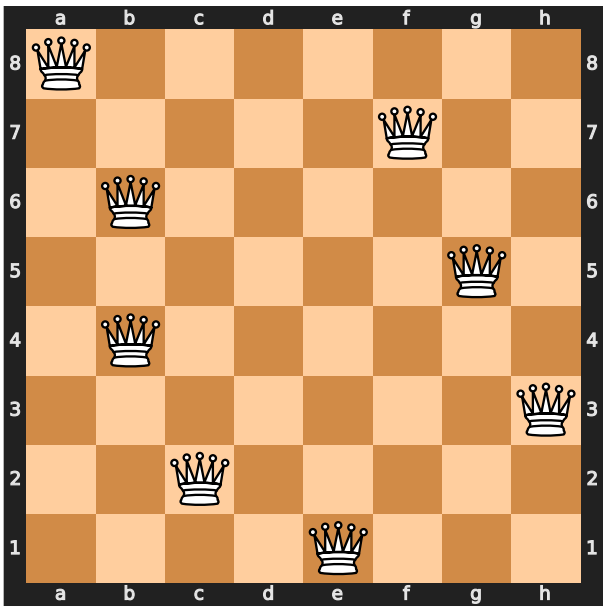
```
    best_particle - the best solution
    scores - the best scores over iterations

    Highly sensitive to the choice of parameters!
    """
    def print_best():
        print(f'iteration {iteration} - best particle: {best_particle}, score: {best_score}')

    solved_cost = np.sum(np.arange(N))
    particles = init_particles(N=N, n_particles=n_particles)
    iteration = 0
    best_particle, best_score = get_best_particle(particles)
    scores = [ best_score ]
    if visualize:
        print('iteration:', iteration)
        show_board(best_particle)
    while best_score < solved_cost and iteration < max_iteration:
        if (iteration % 500) == 0 or iteration == 0:
            print_best()

        for particle in particles:
            rg = np.random.rand((N))
            rp = np.random.rand((N))
            delta_p = particle.best_state - particle.current_particle
            delta_g = best_particle - particle.current_particle
            update = (rp * phip * delta_p +\
                rg * phig * delta_g)     # local vs global
            #print('particle:')
            #print(delta_p)
            #print(delta_g)
            #print(update)
            particle.velocities = omega * particle.velocities + update
            #print(particle.velocities)
            particle.current_particle = (
                np.abs(particle.current_particle + particle.velocities) % N
            ).astype(int)
            # update the particle best
            current_fitness = cost_function(particle.current_particle)
            if current_fitness > particle.best_fitness:
                particle.set_new_best(particle.current_particle, current_fitness)

        best_particle_cand, best_score_cand = get_best_particle(particles)
        if best_score_cand > best_score:
            best_particle, best_score = best_particle_cand, best_score_cand
            if iteration > 0 and visualize:
                print('iteration:', iteration)
                show_board(best_particle)
        scores.append(best_score)
        iteration += 1

    print_best()
    return best_particle, scores
```

In [32]:
```
best_particle, scores = particle_swarm_optimization(
    N=8, omega=1, phip=8, phig=2,
    n_particles=50, visualize=True
)
```

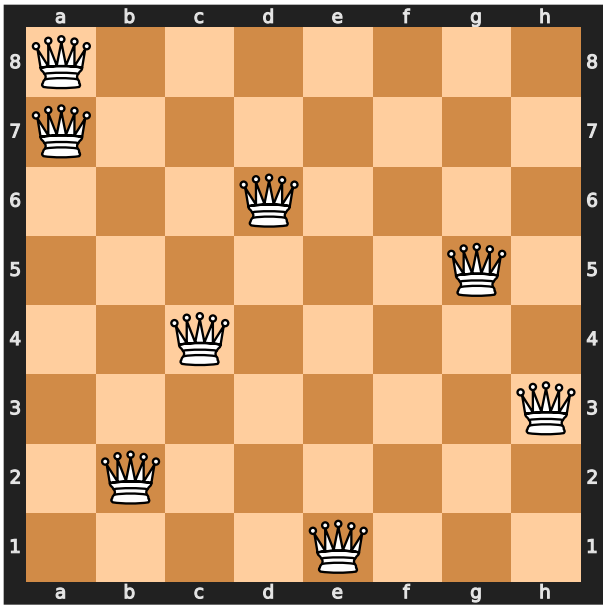
```
iteration: 0
```



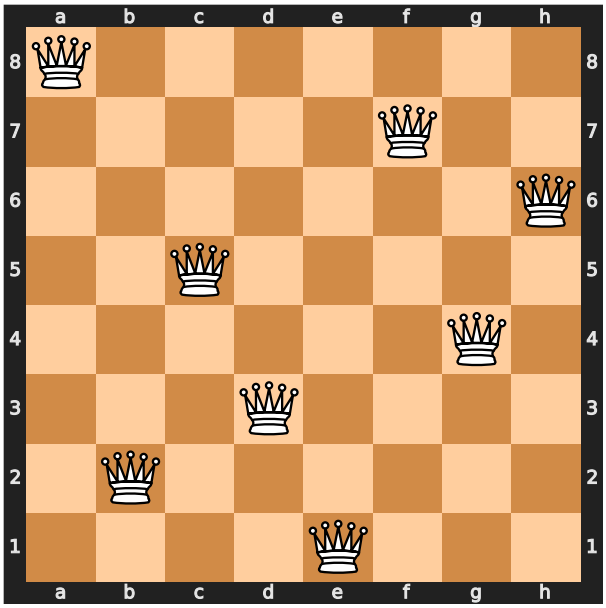
```
iteration 0 - best particle: [2 6 5 2 5 3 0 4], score: 25
iteration: 10
```



```
iteration: 113
```

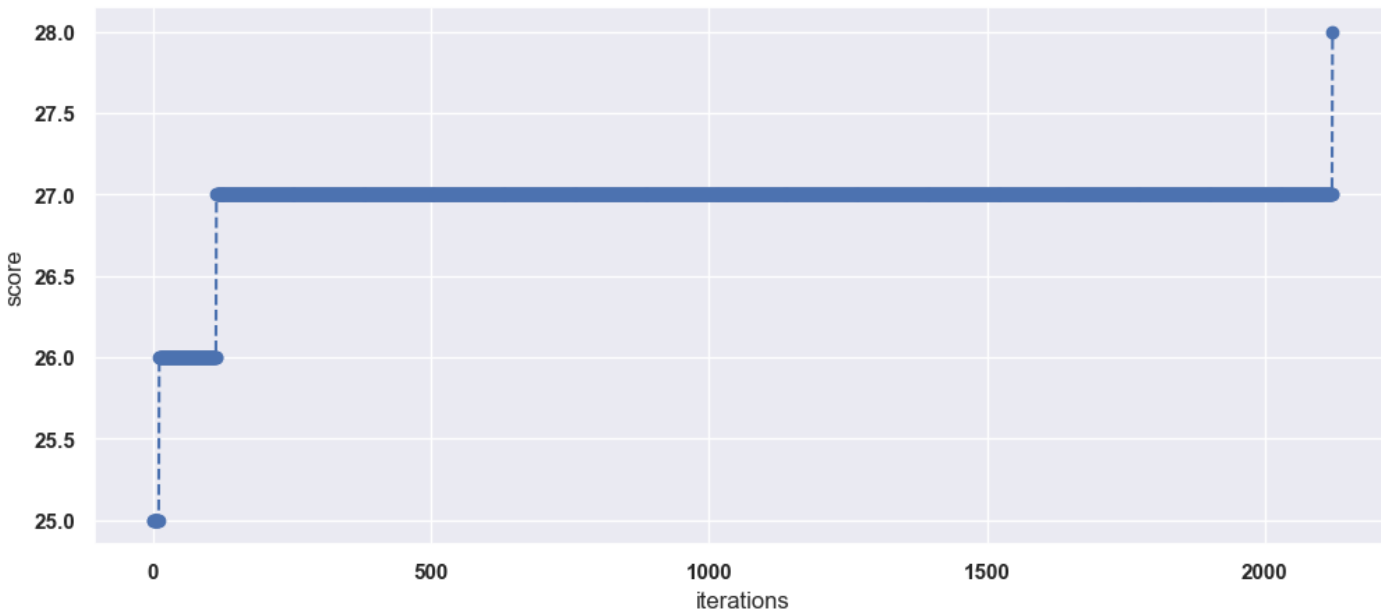


```
iteration 500 - best particle: [0 0 3 6 2 7 1 4], score: 27
iteration 1000 - best particle: [0 0 3 6 2 7 1 4], score: 27
iteration 1500 - best particle: [0 0 3 6 2 7 1 4], score: 27
iteration 2000 - best particle: [0 0 3 6 2 7 1 4], score: 27
iteration: 2121
```




```
iteration 2122 - best particle: [0 5 7 2 6 3 1 4], score: 28
```

In [49]:
```
plot_scores(scores)
```




In [ ]: